

GPUMODE Lecture Study Notes: Lecture 19

Data Processing in GPUs

Core Problem the Lecture Addresses

This lecture studies how to perform **data analytics on GPUs**. The central question is:

How can we take workloads that are traditionally implemented in CPU-based data systems, such as Spark or Pandas, and execute them efficiently on GPU hardware while preserving general-purpose data analytics semantics?

The lecture uses the **one billion row challenge** as a toy workload, then connects that workload to real GPU dataframe execution using **cuDF**, **libcudf**, GPU hash tables, row comparators, gather operations, and a multi-GPU query engine called **Theseus**.

The practical bottleneck is that data analytics workloads often involve extremely large tables, sometimes tens or hundreds of terabytes. CPU systems such as Spark can scale horizontally, but the runtime can asymptote: adding more CPU machines does not always proportionally reduce runtime. GPU systems offer much higher memory bandwidth and massive parallelism, but they require careful design because:

- GPU memory is smaller than cluster CPU memory.
- Tables must be represented in a GPU-friendly columnar format.
- Variable-width data such as strings are difficult to store inside GPU hash maps.
- Dynamic allocation inside kernels is generally avoided.
- General-purpose analytics operators must handle many data types, nulls, strings, multiple keys, multiple values, and multiple aggregations.
- Multi-GPU execution requires partitioning, scheduling, memory estimation, and communication.

The lecture contrasts two approaches:

1. A **custom kernel** written specifically for the one billion row challenge.
2. A **general-purpose dataframe and query engine approach** using cuDF and Theseus.

The custom kernel can exploit the fixed structure of the problem. The general-purpose system is more reusable and extensible, but it must solve harder abstraction problems.

Required Background

Tables, Columns, and Rows

In data analytics, data is usually represented as a **table**. A table consists of columns, and each column stores values of one data type. A row is formed by taking one value from each column at the same logical index.

If a table has n rows and m columns, then column j contains n logical values:

$$C_j = (c_{0j}, c_{1j}, \dots, c_{(n-1)j}).$$

A row i is:

$$R_i = (c_{i0}, c_{i1}, \dots, c_{i(m-1)}).$$

The key property is that all columns in the table have the same number of rows:

$$|C_0| = |C_1| = \dots = |C_{m-1}| = n.$$

Columns may contain fixed-width data such as integers, floating point values, timestamps, and booleans. They may also contain variable-width data such as strings, lists, and structs.

Columnar Representation

GPU analytics systems prefer **columnar storage**. Instead of storing rows contiguously, each column is stored in a separate buffer.

A row-oriented layout looks conceptually like:

$$[(c_{00}, c_{01}, c_{02}), (c_{10}, c_{11}, c_{12}), (c_{20}, c_{21}, c_{22})].$$

A column-oriented layout stores:

$$C_0 = [c_{00}, c_{10}, c_{20}], \quad C_1 = [c_{01}, c_{11}, c_{21}], \quad C_2 = [c_{02}, c_{12}, c_{22}].$$

Columnar storage is important because many analytics queries touch only a subset of columns. It also enables coalesced memory access when many GPU threads read consecutive elements from the same column.

Null Masks

A column may contain null values. In cuDF-style layouts, nullability is usually represented with a **validity bitmask**. The data buffer still stores physical slots, but a null mask determines whether each slot is logically valid.

For a column C , the logical validity of row i is:

$$\text{valid}(i) = \begin{cases} 1, & \text{if bit } i \text{ in the null mask is set,} \\ 0, & \text{otherwise.} \end{cases}$$

A value is logically accessible only if:

$$\text{valid}(i) = 1.$$

This design avoids storing per-element objects and keeps the representation GPU-friendly.

GPU Execution Model

A GPU kernel is launched over a grid of thread blocks. Each block contains many threads. Threads are executed in groups called **warps**. On NVIDIA GPUs, a warp usually contains 32 threads.

A typical one-dimensional mapping is:

$$\text{tid} = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

For a table operation over n rows, a common mapping is:

$$\text{thread tid} \longleftrightarrow \text{row tid}.$$

This mapping is efficient when each thread performs similar work and reads contiguous memory.

cuDF and libcudf

cuDF is a GPU-accelerated dataframe library. It can be understood as a GPU-backed dataframe system with a Python interface similar to Pandas, but with CUDA and C++ implementations underneath.

libcudf is the lower-level C++ library that implements many cuDF operations. Python cuDF calls eventually dispatch into libcudf kernels and device algorithms.

The lecture emphasizes that cuDF provides dataframe APIs such as:

- Reading CSV, JSON, and Parquet files.
- Filtering rows.
- Selecting and rearranging columns.
- Grouping by one or more columns.
- Aggregating with operations such as minimum, maximum, sum, count, and average.
- Performing joins.
- Sorting.
- Dictionary encoding.

Main Concepts

Why Data Analytics on GPUs?

Data analytics workloads often manipulate large tables. These workloads are frequently bandwidth-heavy, parallel, and column-oriented, which makes them plausible candidates for GPU acceleration.

A simplified bandwidth model is:

$$T_{\text{scan}} \approx \frac{B_{\text{input}}}{\text{BW}_{\text{effective}}},$$

where:

- T_{scan} is the time to scan input data.
- B_{input} is the number of bytes read.
- $\text{BW}_{\text{effective}}$ is the effective memory bandwidth achieved by the implementation.

GPUs have much higher memory bandwidth than CPUs, especially when reading columnar data with coalesced accesses. However, high bandwidth is not enough. The system must also solve parsing, grouping, joining, aggregation, memory management, and multi-GPU scheduling.

Pandas, cuDF, and the GPU Dataframe Model

Pandas is widely used for dataframe analytics on CPUs. cuDF aims to provide a similar dataframe abstraction, but backed by GPU memory and CUDA kernels.

A typical cuDF-style workflow is:

```
import cudf

df = cudf.read_csv("measurements.csv", names=["city", "temperature"])

result = (
    df.groupby("city")
    .agg({"temperature": ["min", "max", "mean"]})
)
```

Conceptually, this does three things:

1. Parses the CSV file into GPU-resident columns.
2. Groups rows by the city column.
3. Computes minimum, maximum, and average temperature per city.

The dataframe remains on the GPU after loading. Operations generally produce new dataframe objects rather than mutating the old dataframe. This immutability simplifies reasoning but means memory planning is important because the old and new dataframes may coexist.

Theseus as a Query Engine

cuDF is primarily a single-GPU dataframe library. **Theseus** is described as a GPU-native query or compute engine that can use lower-level cuDF operations as building blocks and scale them across multiple GPUs and multiple nodes.

A SQL query such as:

```
SELECT
    city,
    MIN(temperature) AS min_temperature,
    MAX(temperature) AS max_temperature,
    AVG(temperature) AS avg_temperature
FROM measurements
GROUP BY city
LIMIT 10;
```

is converted into an execution plan. The engine decomposes that plan into tasks such as:

- Read a partition of input data.
- Parse the partition.

- Perform a local group by.
- Concatenate local results.
- Shuffle or hash partition intermediate results across GPUs.
- Perform a final group by.
- Return the final result.

The key idea is that a distributed query engine does not merely call one big kernel. It schedules many GPU tasks while respecting GPU memory constraints.

The One Billion Row Challenge

Problem Definition

The toy workload is the one billion row challenge. The input file contains many lines of the form:

`city;temperature`

Each line contains a city name and a temperature value. The goal is to compute, for each city:

$$\min_c, \quad \max_c, \quad \text{avg}_c.$$

For city c , define the set of row indices belonging to that city as:

$$I_c = \{i \mid \text{city}_i = c\}.$$

The minimum temperature is:

$$\min_c = \min_{i \in I_c} t_i.$$

The maximum temperature is:

$$\max_c = \max_{i \in I_c} t_i.$$

The average temperature is:

$$\text{avg}_c = \frac{\sum_{i \in I_c} t_i}{|I_c|}.$$

A practical implementation often tracks four values per city:

$$(\min_c, \max_c, \text{sum}_c, \text{count}_c).$$

Then:

$$\text{avg}_c = \frac{\text{sum}_c}{\text{count}_c}.$$

Custom GPU Kernel Strategy

The custom solution described in the lecture uses problem-specific assumptions:

- The set of possible cities is known ahead of time.
- The city list is precomputed, sorted, and placed on the GPU.
- A same-sized array stores per-city aggregates.
- The CPU scans the input file to find line boundaries and produces chunk pointers.
- GPU threads process different chunks of the file.
- Each thread parses city and temperature values.
- The thread binary-searches the city in the sorted city list.
- It updates the matching aggregate slot using atomic operations.

A representative sketch is:

```
struct Aggregate {
    float min_value;
    float max_value;
    float sum_value;
    int count;
};

__global__ void process_measurements(
    const char* file_data,
    const size_t* chunk_offsets,
    int num_chunks,
    const CityEntry* sorted_cities,
    int num_cities,
    Aggregate* aggregates
) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;

    if (tid >= num_chunks) {
        return;
    }
}
```

```

size_t start = chunk_offsets[tid];
size_t end = chunk_offsets[tid + 1];

size_t pos = start;

while (pos < end) {
    ParsedLine line = parse_one_line(file_data, pos);

    int city_index = binary_search_city(
        sorted_cities,
        num_cities,
        line.city
    );

    atomicMinFloat(&aggregates[city_index].min_value, line.temperature);
    atomicMaxFloat(&aggregates[city_index].max_value, line.temperature);
    atomicAdd(&aggregates[city_index].sum_value, line.temperature);
    atomicAdd(&aggregates[city_index].count, 1);

    pos = line.next_position;
}
}

```

This kernel is efficient for this exact problem because it is fused: parsing, city lookup, and aggregation happen in one pass over the data.

Why the Custom Kernel Is Not General

The custom kernel is fast because it exploits assumptions that are not true for general analytics systems.

General SQL-style group by must support:

- Unknown key cardinality.
- Multiple key columns.
- Multiple value columns.
- Multiple aggregations.
- Fixed-width and variable-width types.
- Null semantics.
- Strings, lists, structs, dates, and decimals.
- Reusable dataframe outputs.
- Operators beyond group by, such as joins and sorts.

The custom solution also assumes that global memory for all cities and aggregates is known before the kernel starts. GPU kernels generally avoid dynamic global allocation during execution, so general systems pre-allocate storage and use generic data structures.

cuDF Table Representation

Table and Column Objects

A cuDF table consists of multiple columns. Each column owns or references device buffers. A simplified model is:

$$\text{Table} = [C_0, C_1, \dots, C_{m-1}].$$

Each column may have:

- A data buffer.
- A null mask buffer.
- Additional buffers for variable-width data.

For a fixed-width non-null column, the representation is approximately:

$$C = (\text{data}).$$

For a nullable fixed-width column:

$$C = (\text{data}, \text{nullmask}).$$

For strings, a common representation is:

$$C_{\text{string}} = (\text{offsets}, \text{chars}, \text{nullmask}),$$

where:

- $\text{offsets}[i]$ gives the starting byte offset of string i .
- $\text{offsets}[i + 1] - \text{offsets}[i]$ gives the string length.
- chars stores all string bytes contiguously.
- nullmask marks which strings are valid.

Owning Tables and Views

The lecture distinguishes between owning objects and view objects.

An owning column or table owns the device buffers. A view only stores pointers and metadata. A simplified C++ sketch is:

```
struct column {
    device_buffer data;
    device_buffer null_mask;
    size_t size;
    data_type type;
};

struct column_view {
    const void* data;
    const bitmask_type* null_mask;
    size_t size;
    data_type type;
};
```

The owning object is responsible for lifetime management. The view is cheap to copy and pass into algorithms. Because dataframe operations are usually immutable, kernels can safely receive views into existing data and produce new output tables.

Why Views Matter

Views allow operators to avoid copying full data buffers. For example, if a CSV file has many columns but the query only groups by two columns, the system can form a table view containing only those two columns.

If the original table is:

$$T = [C_0, C_1, C_2, C_3, C_4],$$

and the query only needs C_1 and C_3 , the view is:

$$V = [C_1, C_3].$$

No physical column data is copied to create this view. Only metadata and pointers are assembled.

Visual Concept

Draw an owning table object containing several column buffers in GPU memory. Then draw a smaller table view object containing only pointers to selected columns. Emphasize that the view does not copy the underlying data.

cuDF Group By Implementation

Hash-Based Group By

The lecture explains that cuDF group by is hash-based. A hash-based group by maps each input row to a key and stores unique keys in a hash table.

For a group by on key columns K and value column V , the query is conceptually:

GROUP BY K AGG V .

For every row i , define the key:

$$k_i = (K_0[i], K_1[i], \dots, K_{p-1}[i]).$$

Rows i and j belong to the same group if:

$$k_i = k_j.$$

The hash table stores one representative row index per unique key. Instead of storing the entire key directly in the hash table, cuDF stores row indices and uses custom row hashing and row equality operators.

Why Not Store Keys Directly?

Directly storing keys inside a GPU hash map is difficult for general dataframe data.

For fixed-width single-column keys, direct storage is possible. For example, if the key is a 32-bit integer, the hash map can store that integer directly.

But general group by may use:

- Multiple key columns.
- Strings with variable length.
- Nullable values.
- Nested types.

A string key could have arbitrary length. A multi-column key could combine strings, integers, timestamps, and nulls. Pre-allocating enough storage for every hash map slot to hold the largest possible key would waste memory.

Therefore, the hash map stores **row indices**. The actual key values remain in the original table.

Row Equality Comparator

To know whether two row indices represent the same key, cuDF uses a row comparator.

For key table K , the row equality predicate is:

$$\text{equal}(i, j) = \bigwedge_{c=0}^{p-1} (K_c[i] = K_c[j]).$$

This means row i and row j are equal if every key column agrees.

A representative device comparator is:

```
struct row_equal {
    table_device_view keys;

    __device__ bool operator()(size_t lhs, size_t rhs) const {
        for (size_t col = 0; col < keys.num_columns(); ++col) {
            if (!element_equal(keys, col, lhs, rhs)) {
                return false;
            }
        }
        return true;
    }
};
```

The important hardware behavior is that the comparator may read several columns. If the first key column differs, the comparator exits early. This short-circuit behavior can save work, but it may also create divergent control flow if different threads compare rows that fail at different columns.

Row Hash Operator

The hash operator computes a hash value for a row index. For a key row i , the hash is built by hashing each column value and combining those hashes:

$$h(i) = \text{combine}(h_0(K_0[i]), h_1(K_1[i]), \dots, h_{p-1}(K_{p-1}[i])).$$

A representative implementation is:

```
struct row_hash {
    table_device_view keys;

    __device__ size_t operator()(size_t row) const {
        size_t seed = initial_seed();
```

```

        for (size_t col = 0; col < keys.num_columns(); ++col) {
            size_t value_hash = hash_element(keys, col, row);
            seed = hash_combine(seed, value_hash);
        }

        return seed;
    }
};

```

The hash operator allows the hash table to treat a row index as if it were the key. The actual row contents are consulted only through the table device view.

Hash Table Insertion

The hash table is pre-allocated. If the input has n rows, a simple upper bound is that there can be at most n unique groups. Therefore, storage proportional to n is allocated.

Let M be the number of hash table slots. Insertion computes:

$$s_0 = h(i) \bmod M.$$

If slot s_0 is empty, the row index i is inserted. If the slot already contains an equivalent key, insertion returns the existing slot. If the slot contains a different key, the hash table probes another slot.

A simplified linear probing sequence is:

$$s_t = (s_0 + t) \bmod M, \quad t = 0, 1, 2, \dots$$

Empty slots are recognized using a sentinel value. Atomic compare-and-swap is used to claim an empty slot safely.

A simplified insertion sketch is:

```

__device__ size_t insert_or_find(
    size_t row,
    size_t* slots,
    size_t num_slots,
    row_hash hasher,
    row_equal equal
) {
    size_t hash_value = hasher(row);
    size_t slot = hash_value % num_slots;

    while (true) {
        size_t old = atomicCAS(

```

```

        &slots[slot],
        EMPTY_SENTINEL,
        row
    );

    if (old == EMPTY_SENTINEL) {
        return slot;
    }

    if (equal(old, row)) {
        return slot;
    }

    slot = (slot + 1) % num_slots;
}
}

```

This is the core mechanism that turns row indices into group identifiers.

Aggregation with Atomics

Once a row has been mapped to a hash table slot, that slot identifies the group. The aggregation arrays are indexed by slot.

For row i , suppose insertion returns slot s . Then the value x_i updates:

$$\text{sum}[s] \leftarrow \text{sum}[s] + x_i,$$

$$\text{count}[s] \leftarrow \text{count}[s] + 1,$$

$$\text{min}[s] \leftarrow \min(\text{min}[s], x_i),$$

$$\text{max}[s] \leftarrow \max(\text{max}[s], x_i).$$

In parallel code, these updates use atomics:

```

size_t slot = insert_or_find(
    row,
    slots,
    num_slots,
    hasher,
    equal
);

```

```

atomicAdd(&sum_values[slot], temperature[row]);
atomicAdd(&count_values[slot], 1);
atomicMinFloat(&min_values[slot], temperature[row]);
atomicMaxFloat(&max_values[slot], temperature[row]);

```

Atomics are necessary because many rows can map to the same group simultaneously.

Sparse Intermediate Output

The hash table may have many empty slots. After insertion and aggregation, the data is sparse:

$$\text{slots} = [\text{empty}, r_5, \text{empty}, r_{20}, r_7, \dots].$$

The aggregate arrays are sparse in the same way:

$$\text{sum} = [\text{unused}, \text{sum}_{r_5}, \text{unused}, \text{sum}_{r_{20}}, \text{sum}_{r_7}, \dots].$$

The system must compact the non-empty slots. This can be done with operations like `copy_if`, prefix sums, or other stream compaction primitives.

If $E[s]$ indicates whether slot s is occupied:

$$E[s] = \begin{cases} 1, & \text{if slot } s \text{ is occupied,} \\ 0, & \text{otherwise.} \end{cases}$$

The compacted position of slot s is given by a prefix sum:

$$p[s] = \sum_{u=0}^{s-1} E[u].$$

Occupied slot s moves to position $p[s]$.

Gathering the Group Keys

After compaction, the system has aggregate values for each unique group. But it also needs to return the corresponding key values.

The hash table stores representative row indices. If compacted slot q stores representative row r_q , then the output key row is gathered from the original key table:

$$K_{\text{out}}[q] = K_{\text{input}}[r_q].$$

This operation is called **gather**.

For a gather map G , the output column is:

$$C_{\text{out}}[i] = C_{\text{in}}[G[i]].$$

Gather is a fundamental primitive because it materializes the final result after the system has worked with indices for as long as possible.

```
for_each_output_row(i) {  
    size_t source_row = gather_map[i];  
  
    output_city[i] = input_city[source_row];  
    output_min[i] = compacted_min[i];  
    output_max[i] = compacted_max[i];  
    output_sum[i] = compacted_sum[i];  
    output_count[i] = compacted_count[i];  
}
```

For strings, gather is more complicated because the output must copy variable-length byte ranges. cuDF can specialize string gather so that large strings are copied using blocks and smaller strings are copied using threads.

Visual Concept

Draw an input table on the left, a hash table in the middle storing row indices, and an output table on the right. Show arrows from compacted hash slots back to the original input rows. Label the final step as gather.

Hardware-Level Explanation

Global Memory

The table data lives in GPU global memory. For fixed-width columns, consecutive rows are stored contiguously:

$$\text{addr}(C[i]) = \text{base}(C) + i \cdot \text{sizeof}(T).$$

If thread t reads $C[t]$, then consecutive threads read consecutive memory addresses. This is ideal for coalescing.

For a warp of 32 threads, the addresses are:

$$\text{base}(C), \text{base}(C) + \text{sizeof}(T), \dots, \text{base}(C) + 31 \cdot \text{sizeof}(T).$$

This pattern can be served efficiently by the memory system.

Strings and Variable-Width Data

Strings break the simplicity of fixed-width access. A string column typically requires at least two memory accesses:

1. Read offsets:

$$a = \text{offsets}[i], \quad b = \text{offsets}[i + 1].$$

2. Read bytes:

$$\text{chars}[a : b].$$

The string length is:

$$\ell_i = b - a.$$

Different rows can have different string lengths. This causes irregular memory access and variable work per thread.

Hash Table Contention

Hash table operations are irregular. Even if input row access is coalesced, hash table slots are pseudo-random due to hashing:

$$s = h(i) \bmod M.$$

Threads in the same warp may access unrelated slots. This reduces coalescing and can increase memory latency.

Additionally, hot keys can cause atomic contention. If many rows belong to the same city, many threads update the same aggregate slot.

For a key c , the approximate contention is proportional to:

$$\text{contention}(c) \propto |I_c|.$$

High-cardinality data spreads updates across many slots. Low-cardinality data can cause heavy contention because many rows share the same few keys.

Occupancy and Temporary Memory

Group by requires temporary storage:

- Hash table slots.
- Aggregate buffers.
- Null masks.
- Device views.

- Compaction buffers.
- Gather maps.

A simple upper bound for hash table storage is:

$$B_{\text{hash}} \approx M \cdot \text{sizeof}(\text{slot}).$$

If $M \approx \alpha n$, where α is an over-allocation factor, then:

$$B_{\text{hash}} \approx \alpha n \cdot \text{sizeof}(\text{slot}).$$

For aggregate storage with a aggregate buffers of width w :

$$B_{\text{agg}} \approx a \cdot M \cdot w.$$

The total temporary memory can be substantial:

$$B_{\text{temp}} \approx B_{\text{hash}} + B_{\text{agg}} + B_{\text{compaction}} + B_{\text{metadata}}.$$

This is why query engines must estimate memory before launching tasks.

Kernel Fusion Versus Generality

The custom one billion row challenge kernel fuses parsing, lookup, and aggregation. Fusion reduces global memory traffic:

$$B_{\text{fused}} < B_{\text{parse}} + B_{\text{materialize}} + B_{\text{groupby}}.$$

However, fusion reduces generality. A dataframe library often materializes intermediate columns because later operations may reuse them.

The trade-off is:

custom fused kernel $\Rightarrow$ fast but narrow,

general dataframe operators $\Rightarrow$ reusable but more memory movement.

Mathematical Reasoning

Group By Semantics

Let X be the input table, K be the set of key columns, and V be a value column. The group by partitions rows into equivalence classes.

Rows i and j are equivalent if:

$$i \sim j \iff K[i] = K[j].$$

Each group is an equivalence class:

$$G_k = \{i \mid K[i] = k\}.$$

For an aggregation function f , the output is:

$$Y[k] = f(\{V[i] \mid i \in G_k\}).$$

For multiple aggregations:

$$Y[k] = \left(\min_{i \in G_k} V[i], \max_{i \in G_k} V[i], \sum_{i \in G_k} V[i], |G_k| \right).$$

Average is derived from sum and count:

$$\text{avg}(k) = \frac{\sum_{i \in G_k} V[i]}{|G_k|}.$$

Hash Table Load Factor

The load factor of a hash table is:

$$\lambda = \frac{U}{M},$$

where:

- U is the number of unique keys.
- M is the number of hash table slots.

As λ approaches 1, collisions increase. Therefore, practical hash tables allocate more slots than the expected number of unique keys.

The expected probe length depends on the collision strategy and load factor. The important intuition is:

$\lambda \uparrow \Rightarrow$ more probes $\Rightarrow$ more random memory access $\Rightarrow$ lower throughput.

Arithmetic Intensity

Arithmetic intensity is:

$$\text{AI} = \frac{\text{FLOPs}}{\text{bytes moved}}.$$

Many data analytics operators have low arithmetic intensity. A simple scan with a predicate may perform only a few operations per loaded value. Group by can perform more work, but much of that work is memory-system dominated due to hash table accesses, string reads, and atomics.

The roofline model says:

$$P \leq \min(P_{\text{peak}}, \text{AI} \cdot B_{\text{peak}}),$$

where:

- P is achieved performance.
- P_{peak} is peak compute throughput.
- B_{peak} is peak memory bandwidth.
- AI is arithmetic intensity.

For low arithmetic intensity workloads:

$$\text{AI} \cdot B_{\text{peak}} < P_{\text{peak}},$$

so the workload is bandwidth-bound or memory-latency-bound.

Multi-GPU Hash Partitioning

For multi-GPU group by, intermediate grouped results must be redistributed so that all rows for the same key land on the same GPU.

If there are P GPUs, a partition function can be:

$$p(k) = h(k) \bmod P.$$

Each GPU sends group rows with partition p to GPU p . After exchange, GPU p owns all keys satisfying:

$$h(k) \bmod P = p.$$

Then each GPU performs a final local group by.

This is necessary because local group by on each GPU only produces partial results. If city c appears on multiple GPUs, its partial aggregates must be combined.

For sums and counts:

$$\text{sum}_c = \sum_{g=0}^{P-1} \text{sum}_{c,g},$$

$$\text{count}_c = \sum_{g=0}^{P-1} \text{count}_{c,g}.$$

For minimum and maximum:

$$\min_c = \min_g \min_{c,g},$$

$$\max_c = \max_g \max_{c,g}.$$

Then:

$$\text{avg}_c = \frac{\sum_{g=0}^{P-1} \text{sum}_{c,g}}{\sum_{g=0}^{P-1} \text{count}_{c,g}}.$$

Code Walkthrough

Pandas-Style cuDF Solution

The high-level cuDF version is concise:

```

import cudf

df = cudf.read_csv(
    "measurements.csv",
    sep=";",
    names=["city", "temperature"]
)

result = (
    df.groupby("city")
    .agg({
        "temperature": ["min", "max", "mean"]
    })
)

```

Line-by-line explanation:

- `cudf.read_csv` parses the CSV and creates GPU-resident columns.
- The city column is variable-width string data.
- The temperature column is fixed-width numeric data.
- `groupby("city")` creates groups based on city equality.
- `agg` requests multiple aggregations over the temperature column.
- Internally, this requires hashing city rows, aggregating values, compacting hash table output, and gathering group keys.

The simplicity of the Python code hides the complexity of GPU data structures and kernels underneath.

SQL Query in Theseus

The query engine version is:

```

SELECT
    city,
    MIN(temperature) AS min_temperature,
    MAX(temperature) AS max_temperature,
    AVG(temperature) AS avg_temperature
FROM measurements
GROUP BY city
LIMIT 10;

```

This SQL query is declarative. The user states the desired result, not the execution strategy. Theseus translates it into a physical plan.

A possible single-GPU plan is:

1. Split the CSV file into chunks.

2. For each chunk, read and parse the chunk.
3. Run a local group by on the chunk.
4. Concatenate all local grouped results.
5. Run another group by to merge partial aggregates.
6. Return the final rows.

A possible multi-GPU plan is:

1. Assign input chunks across GPUs.
2. Each GPU performs local CSV read and local group by.
3. Each GPU hash partitions its partial results by city.
4. GPUs exchange partitions.
5. Each GPU concatenates received partitions.
6. Each GPU performs final group by on its assigned key partition.
7. Results are collected for the user.

Representative Local Group By Kernel

A simplified group by kernel can be written as:

```
__global__ void groupby_kernel(
    table_device_view key_columns,
    column_device_view<float> values,
    size_t num_rows,
    size_t* hash_slots,
    float* min_values,
    float* max_values,
    float* sum_values,
    int* count_values,
    size_t num_slots
) {
    size_t row = blockIdx.x * blockDim.x + threadIdx.x;

    if (row >= num_rows) {
        return;
    }

    row_hash hasher{key_columns};
    row_equal equal{key_columns};

    size_t slot = insert_or_find(
        row,
```

```

        hash_slots,
        num_slots,
        hasher,
        equal
    );

    float x = values.element(row);

    atomicMinFloat(&min_values[slot], x);
    atomicMaxFloat(&max_values[slot], x);
    atomicAdd(&sum_values[slot], x);
    atomicAdd(&count_values[slot], 1);
}

```

This representative kernel omits many production details, but it captures the lecture's core idea.

Important hardware behavior:

- The row mapping is embarrassingly parallel at first.
- Key hashing can involve multiple columns.
- String keys introduce variable memory access.
- Hash table insertion produces random memory accesses.
- Atomics serialize updates when many rows share a key.
- Output is sparse and must be compacted.

Representative Gather Operation

A simplified gather for fixed-width columns is:

```

__global__ void gather_float_kernel(
    const float* input,
    const size_t* gather_map,
    float* output,
    size_t num_output_rows
) {
    size_t row = blockIdx.x * blockDim.x + threadIdx.x;

    if (row >= num_output_rows) {
        return;
    }

    size_t source_row = gather_map[row];
    output[row] = input[source_row];
}

```


For fixed-width data, gather is straightforward. For strings, gather must rebuild offsets and copy byte ranges.

Representative Hash Partition

A simplified hash partition step is:

```
__global__ void compute_partitions(
    table_device_view keys,
    size_t num_rows,
    int num_partitions,
    int* partition_ids
) {
    size_t row = blockIdx.x * blockDim.x + threadIdx.x;

    if (row >= num_rows) {
        return;
    }

    row_hash hasher{keys};
    size_t h = hasher(row);

    partition_ids[row] = static_cast<int>(h % num_partitions);
}
```

This produces a partition id for each row. A later step rearranges rows so that rows for partition p can be sent to GPU p .

Performance Intuition

Why the Custom Kernel Can Be Fast

The custom one billion row challenge kernel can be fast because:

- It makes only one pass through the data.
- It avoids materializing a general dataframe.
- It knows the schema in advance.
- It knows the aggregation pattern in advance.
- It can keep temporary parsing state in registers.
- It directly updates final aggregate arrays.

The approximate memory traffic is closer to:

$$B_{\text{custom}} \approx B_{\text{file}} + B_{\text{aggregate updates}}.$$

Why the General cuDF Version Is Still Valuable

The general cuDF version is valuable because it supports a much broader class of queries.

Its memory traffic may look more like:

$$B_{\text{general}} \approx B_{\text{file read}} + B_{\text{parsed columns}} + B_{\text{hash table}} + B_{\text{aggregate buffers}} + B_{\text{compaction}} + B_{\text{gather}}.$$

This is more expensive, but the result is reusable and composable.

Memory-Bound Versus Compute-Bound Behavior

Many analytics operators are not limited by floating point throughput. They are limited by:

- Memory bandwidth.
- Memory latency.
- Random access patterns.
- Atomic contention.
- String processing irregularity.
- Temporary allocation and compaction overhead.
- Data transfer across GPUs or nodes.

A group by with strings and hash tables may be memory-latency-heavy rather than pure bandwidth-bound because accesses are irregular.

Why Materializing Late Helps

The lecture emphasizes that cuDF utilities often move indices around and materialize final data only at the end. This is powerful because fixed-width indices are easier to sort, copy, shuffle, and compact than full rows or variable-width strings.

Instead of physically moving strings many times, the system moves row indices:

$$\text{expensive movement of strings} \longrightarrow \text{cheap movement of indices.}$$

Only at the final gather step are the actual strings copied into the output table.

Abstractions Can Be Fast

A major engineering lesson from the lecture is that reusable GPU data structures can be competitive with highly specialized kernels. The speaker describes replacing a complicated hand-tuned Parquet dictionary encoding kernel with a solution built using reusable GPU collection abstractions. The new version was not slower and was much more extensible.

The lesson is:

For large GPU systems work, the right abstraction can be more valuable than manually writing every low-level trick yourself.

Libraries such as CUB, Thrust, libcudf, libcu++, and cuCollections provide reusable building blocks that can reduce code complexity while preserving high performance.

Dictionary Encoding

Motivation

Dictionary encoding is used when a column has low cardinality. Instead of storing every value repeatedly, the system stores:

- A dictionary of unique values.
- An index column pointing into the dictionary.

If the original column is:

$$C = [\text{red}, \text{blue}, \text{red}, \text{green}, \text{blue}] ,$$

then dictionary encoding stores:

$$D = [\text{red}, \text{blue}, \text{green}] ,$$

$$I = [0, 1, 0, 2, 1] .$$

The reconstructed value is:

$$C[i] = D[I[i]] .$$

Connection to Hash Maps

Dictionary encoding requires finding unique values and assigning ids. This is naturally hash-table-based:

$$\text{value} \longrightarrow \text{dictionary id.}$$

The lecture notes that hash maps used for group by are also useful for dictionary encoding. This is especially relevant for Parquet writing, where dictionary encoding can reduce file size.

Joins

Equality Joins

The lecture briefly mentions joins. Equality joins can also use hash tables. For two tables A and B , an equality join on key K finds pairs:

$$(i, j) \text{ such that } K_A[i] = K_B[j].$$

A common hash join strategy is:

1. Build a hash table from one side of the join.
2. Probe the hash table using rows from the other side.
3. Emit matching row pairs.

For distributed or multi-GPU joins, both tables may be hash partitioned by the join key so that matching keys land on the same GPU.

The partition function is:

$$p(k) = h(k) \bmod P.$$

After partitioning, GPU p performs the join for keys assigned to partition p .

Inequality Joins

The speaker notes that inequality joins are much harder. An inequality join may require conditions such as:

$$A.x < B.y.$$

Hashing works well for equality because equal keys have the same hash. Inequality relationships do not map as directly to hash tables. They may require sorting, range partitioning, interval structures, or more specialized algorithms.

Theseus Multi-GPU Execution

Task Decomposition

Theseus decomposes a query plan into tasks. Each task carries estimates such as:

- Input size.
- Expected output size.
- Required temporary memory.
- Operating bytes.

The scheduler launches a task only when enough GPU memory is available.

This is essential because GPU memory is limited. Even when a GPU has high bandwidth, launching too many memory-heavy tasks can cause out-of-memory failures or force spilling.

Single-GPU Chunked Execution

For a large CSV file, Theseus may divide the file into many chunks. Each chunk becomes a task:

$$\text{chunk}_i \longrightarrow \text{CSV read} \longrightarrow \text{local group by}.$$

If there are 100 chunks, the plan may run 100 read-plus-group-by tasks. Each local result is much smaller than the input chunk because the number of cities is limited.

Then the engine concatenates the partial results:

$$R_{\text{concat}} = R_0 \cup R_1 \cup \dots \cup R_{99}.$$

Finally, it performs another group by:

$$R_{\text{final}} = \text{GroupBy}(R_{\text{concat}}).$$

Multi-GPU Execution

With P GPUs, input tasks are distributed across devices. Each GPU computes local partial aggregates. Then partial results are hash partitioned by key and exchanged.

For GPU g , let its local partial result be R_g . It partitions:

$$R_g = R_{g,0} \cup R_{g,1} \cup \dots \cup R_{g,P-1},$$

where $R_{g,p}$ contains rows whose key belongs to partition p .

GPU p receives:

$$S_p = R_{0,p} \cup R_{1,p} \cup \dots \cup R_{P-1,p}.$$

Then GPU p performs the final group by over S_p :

$$F_p = \text{GroupBy}(S_p).$$

The complete result is:

$$F = F_0 \cup F_1 \cup \dots \cup F_{P-1}.$$

Visual Concept

Draw two GPUs side by side. Each GPU reads a different half of the input file and performs a local group by. Then draw a hash partition exchange where each GPU sends part of its grouped result to the other GPU. Finally, each GPU performs a final group by on the partition it owns.

Common Misconceptions

Misconception: GPU Data Analytics Is Just Pandas on a GPU

cuDF provides a Pandas-like interface, but the implementation is fundamentally different. It requires GPU memory layouts, CUDA kernels, device-side hash tables, null masks, string buffers, and specialized algorithms for gather, sort, join, and group by.

Misconception: The Fastest Solution Is Always the Best General Solution

A custom fused kernel may win on one benchmark, but it may not support general SQL semantics. A dataframe library must work across many schemas, types, null patterns, and query shapes.

Misconception: Group By Stores Full Keys in the Hash Table

For general dataframe group by, cuDF does not need to store the full multi-column key directly in the hash table. It can store row indices and use row hashing plus row equality to interpret those indices as keys.

Misconception: Strings Behave Like Integers

Strings are variable-width. They require offsets, byte buffers, and specialized copy logic. A string comparison may read many bytes and can create irregular control flow.

Misconception: More GPUs Automatically Solve the Problem

Multi-GPU execution requires partitioning and communication. If partitioning is imbalanced or communication dominates, scaling can suffer. The engine must schedule tasks carefully and manage memory pressure.

Misconception: Low-Level Assembly Is Always Required

The lecture pushes against the idea that every GPU optimization requires hand-written PTX or SASS. In many data analytics systems, high-quality abstractions such as CUB, Thrust, cuCollections, libcudf, and libcu++ can produce competitive implementations.

Practical Takeaways

- GPU analytics is powerful because table operations often expose row-level and column-level parallelism.
- Columnar memory layouts are central to GPU dataframe performance.
- cuDF is a single-GPU dataframe library with a Python front end and CUDA/C++ back end.
- libcudf provides lower-level C++ operators that query engines can call directly.
- Theseus scales GPU dataframe-style operations across multiple GPUs and nodes.
- General group by is harder than a toy benchmark because it must support multiple key columns, strings, nulls, and arbitrary aggregations.
- cuDF group by uses hash tables, row comparators, row hashers, atomics, compaction, and gather.
- Storing row indices in the hash table avoids storing arbitrary variable-width keys inside the hash table.
- Gather is a central primitive because many algorithms move indices first and materialize data later.
- Hash partitioning is the key idea for distributed group by and equality joins.

- GPU query engines must reason about memory before launching work.
- Reusable CUDA abstractions can be both productive and performant.

Project or Experiment Ideas

Naive Versus cuDF Group By

Implement the one billion row challenge in three ways:

1. Pandas on CPU.
2. cuDF on a single GPU.
3. A custom CUDA kernel for a simplified fixed-schema version.

Measure:

$$T_{\text{pandas}}, \quad T_{\text{cuDF}}, \quad T_{\text{custom}}.$$

Compare the engineering complexity and performance.

Hash Table Load Factor Experiment

Build a simple GPU hash table and vary the load factor:

$$\lambda = \frac{U}{M}.$$

Benchmark insertion throughput as λ changes. Plot throughput versus load factor and observe collision behavior.

String Versus Integer Group By

Create two datasets:

- One with integer keys.
- One with string keys.

Run group by on both and compare runtime. The goal is to observe how variable-width keys affect hashing, comparison, and gather.

Atomic Contention Experiment

Generate datasets with different key cardinalities:

- Very low cardinality, such as 10 unique keys.
- Medium cardinality, such as 10^4 unique keys.

- High cardinality, such as 10^7 unique keys.

Measure group by runtime. Low cardinality may increase atomic contention. High cardinality may increase hash table size and random access.

Late Materialization Experiment

Implement two versions of a simple sorting or grouping pipeline:

1. Move full rows at every step.
2. Move indices and gather only at the end.

Measure memory traffic and runtime. This experiment demonstrates why dataframe systems prefer to manipulate indices before materializing final outputs.

Multi-GPU Hash Partition Simulation

Even without multiple GPUs, simulate partitioning on one GPU or CPU. Given a set of keys, compute:

$$p(k) = h(k) \bmod P.$$

Measure partition balance for different key distributions. This helps explain why distributed group by and joins depend on good hash partitioning.

Final Mental Model

GPU data analytics is not merely about running Pandas code on faster hardware. It is about representing tables in GPU-friendly columnar buffers, expressing dataframe operations as parallel kernels and device algorithms, and building reusable primitives such as hash tables, row comparators, row hashers, compaction, and gather.

The one billion row challenge shows the appeal of a custom fused GPU kernel: one pass, direct parsing, and direct aggregation. But production analytics systems need generality. cuDF solves this by building general-purpose GPU dataframe operators. Theseus scales those operators by converting SQL into tasks, estimating memory, scheduling work across GPUs, hash partitioning intermediate results, and merging partial aggregates.

The deepest mental model is:

A GPU query engine keeps data columnar, moves indices instead of full rows whenever possible, uses hash-based primitives to discover equivalence classes, uses atomics and compaction to build grouped results, and delays materialization until gather produces the final table.